

PROJECT REPORT

AI-Powered Data Exfiltration Detection and User Risk Profiling System

Programming in Java

Project Domain	Cybersecurity / AI & Machine Learning
Project Type	Java-based Modular Application
Primary Function	Data Exfiltration Detection and User Risk Profiling
Data Format	CSV
Version Control	Git / GitHub

Abstract

The AI-Powered Data Exfiltration Detection and User Risk Profiling System is a Java-based cybersecurity application designed to analyse user activity and identify behaviour that may indicate unauthorized data transfer. The system follows a modular pipeline consisting of dataset loading, preprocessing, feature engineering, behaviour profiling, anomaly detection, threat classification, risk scoring, explainability, and alert generation.

The project separates raw data from source code and is designed for maintainability, validation, testing, documentation, and future extension. A controlled synthetic dataset may be used for experimentation; any synthetic data must be clearly identified as synthetic. Real cybersecurity datasets can be incorporated in an advanced version when access and licensing permit.

Keywords

Data Exfiltration, Cybersecurity, User Behaviour Analytics, Anomaly Detection, Risk Profiling, Java, Machine Learning, Explainable AI

1. Introduction

Organizations generate large volumes of user, file, session, and network activity. Among the security problems that can be investigated using such information is data exfiltration, meaning unauthorized movement of information from a protected environment to an external destination.

The proposed system provides a structured workflow for analysing activity rather than relying on a single indicator. It prepares the data, derives relevant features, establishes user behaviour profiles, detects deviations, classifies potentially risky activity, calculates risk, explains the result, and generates alerts.

2. Problem Statement

Manual analysis of large activity logs can be time-consuming and can make it difficult to identify unusual combinations of events. Indicators such as unusually large outbound transfers, excessive file access, unusual login times, repeated external communication, and deviations from a user's normal behaviour may require further investigation.

This project addresses the problem by creating a modular Java application that processes activity records and produces security-oriented outputs including activity classification, risk score, explanations, and alerts.

3. Objectives

- Load and validate user activity records from structured datasets.
- Preprocess raw records and create analysis-ready data.
- Engineer security-relevant behavioural and transfer features.
- Build user behaviour profiles and behavioural baselines.
- Detect anomalous activity using the implemented detection approach.
- Classify activity as NORMAL, SUSPICIOUS, or EXFILTRATION.
- Calculate a 0–100 risk score and corresponding risk level.
- Generate human-readable explanations and security alerts.
- Evaluate the implemented approach using actual test results.

4. Scope and Target Users

Scope: CSV-based data ingestion, preprocessing, feature engineering, behaviour profiling, anomaly detection, classification, risk scoring, explainability, alert generation, testing, and results storage. Future extensions may include real-time monitoring, database storage, dashboards, and additional machine-learning methods.

Target users: security analysts, system administrators, cybersecurity students/researchers, and academic project evaluators.

5. Functional Requirements

ID	Module	Input	Output
FR-01	Data Input & Preprocessing	CSV records	Validated records
FR-02	Feature Engineering	Processed records	Security features
FR-03	Behaviour Profiling	User history	User baseline
FR-04	Anomaly Detection	Feature vectors	Anomaly result
FR-05	Threat Classification	Activity features	Threat class
FR-06	Risk Scoring	Detection indicators	Score + level
FR-07	Explainability	Risk factors	Reasons
FR-08	Alert Generation	Risk/classification	Security alert

6. Non-Functional Requirements

Requirement	Description
Performance	Process academic-scale activity datasets efficiently.
Security	Validate inputs and avoid hard-coded sensitive credentials.
Usability	Provide understandable risk levels, explanations, and alerts.
Reliability	Handle malformed data and runtime errors safely.
Scalability	Allow larger datasets and improved algorithms to be introduced.
Maintainability	Separate data, processing, analytics, risk, alert, and evaluation logic.
Error Handling	Validate file availability, record structure, and numeric fields.
Logging	Record important processing and security events where implemented.

7. System Architecture

The system follows the end-to-end pipeline below:

Dataset → Preprocessing → Feature Engineering → Behaviour Profiling → Anomaly Detection → Classification → Risk Scoring → Explainability → Alert Generation

Architecture Layers

Layer	Main Components	Purpose
Data	activity.csv, processed data	Store and provide activity records.
Processing	DatasetLoader, DataPreprocessor, FeatureEngineer	Validate and transform data.
Analytics	BehaviourProfiler, AnomalyDetector, ThreatClassifier	Model behaviour and identify/classify activity.
Risk & Response	RiskScorer, ExplainabilityEngine, AlertGenerator	Quantify, explain, and report risk.
Evaluation	ModelEvaluator, results	Measure actual system performance.

8. Project Modules

- **DatasetLoader:** Reads CSV records and converts them into ActivityRecord objects.
- **DataPreprocessor:** Validates fields, ranges, and malformed records.
- **FeatureEngineer:** Derives security-related analysis features.
- **BehaviourProfiler:** Builds user-level behavioural baselines.
- **AnomalyDetector:** Identifies unusual activity using the implemented detection approach.
- **ThreatClassifier:** Produces NORMAL, SUSPICIOUS, or EXFILTRATION classification.
- **RiskScorer:** Produces a bounded 0–100 risk score and risk level.
- **ExplainabilityEngine:** Produces human-readable reasons for detected risk.
- **AlertGenerator:** Creates security alerts.
- **ModelEvaluator:** Calculates evaluation metrics from actual predictions.

9. Repository Structure

The recommended repository keeps README.md and statement.md at the root, separates data/raw from data/processed, and organizes Java classes under package directories such as data, preprocessing, features, profiling, anomaly, classification, risk, explainability, alert, and evaluation.

10. Dataset Description

The activity dataset represents user behaviour and transfer characteristics. A controlled synthetic dataset can be used for academic experimentation. Synthetic records must not be represented as data taken from a real cybersecurity dataset.

Field	Description
user_id	User identifier for an activity record.
timestamp	Date and time of activity.
login_hour	Hour associated with login/activity.
session_duration	Session duration.
files_accessed	Number of files accessed.
bytes_sent	Outbound data volume.
bytes_received	Inbound data volume.
packet_count	Network packet activity indicator.
destination_frequency	Frequency of destination communication.
transfer_frequency	Frequency of data transfers.
file_access_frequency	Frequency of file access.
baseline_deviation	Deviation from established behaviour.
protocol	Communication protocol.
destination_type	Internal/external/unknown destination.
label	NORMAL, SUSPICIOUS, or EXFILTRATION.

11. Feature Engineering

Feature engineering transforms raw activity information into indicators suitable for behavioural and security analysis.

- Temporal features: login hour and timestamp-derived information.
- Volume features: bytes sent, bytes received, and packet count.
- Access features: files accessed and file-access frequency.
- Communication features: destination and transfer frequency.
- Behavioural feature: baseline deviation.
- Context features: protocol and destination type.

12. Detection, Classification and Risk Design

The anomaly-detection stage identifies activity that differs from expected behaviour. The classification stage maps activity evidence to the project categories NORMAL, SUSPICIOUS, and EXFILTRATION. A trained machine-learning classifier can be used when appropriate training and evaluation data are available.

Prototype Risk Levels

Risk Score	Level
0–24	LOW
25–49	MEDIUM
50–74	HIGH
75–100	CRITICAL

The prototype risk calculation can combine indicators such as large outbound transfer, excessive file access, unknown external destination, high transfer frequency, baseline deviation, unusual login time, packet count, and external communication.

13. Design Decisions

- CSV is used for the initial dataset because it is portable, readable, and suitable for academic experimentation.
- Raw data and source code are kept separate to improve project organization.

- Package-based Java modules separate responsibilities and improve maintainability.
- Risk and explainability are independent modules so scoring and explanations can evolve separately.
- A dedicated evaluation module keeps metric calculation separate from detection logic.
- Git/GitHub provides version control and supports reproducible project organization.

14. Key Classes

Main.java coordinates the pipeline. ActivityRecord represents one activity. DatasetLoader loads data. DataPreprocessor validates records. FeatureEngineer creates derived features. UserProfile and BehaviourProfiler maintain baselines. AnomalyDetector and ThreatClassifier perform detection/classification. RiskScorer, ExplainabilityEngine, AlertGenerator, and ModelEvaluator complete the risk, response, and evaluation stages.

15. Testing Approach

Testing should verify individual modules as well as the complete workflow. The project should include JUnit tests for normal inputs, malformed CSV records, invalid numeric values, empty datasets, boundary risk scores, anomaly detection, and alert generation.

Test Area	Verification
Dataset loading	Valid CSV loads; malformed rows are handled safely.
Preprocessing	Missing/invalid fields are handled according to defined rules.
Feature engineering	Derived values are generated consistently.
Behaviour profiling	User baseline is created from available activity.
Anomaly detection	Representative feature vectors return valid detection results.
Risk scoring	Score remains between 0 and 100 and maps correctly to a level.
Alert generation	High-risk activity produces an alert with relevant information.
End-to-end	Valid input passes through the complete pipeline.

Run the automated tests with: **mvn test**.

16. Evaluation Methodology

When a supervised classifier is implemented, training and evaluation data should be separated and predictions should be compared with ground-truth labels. The system can report Precision, Recall, F1 Score, and Accuracy. Numerical values should be inserted only after the actual project has been executed and evaluated.

Metric	Meaning
Precision	Correctness among predicted positive cases.
Recall	Proportion of actual positive cases detected.
F1 Score	Harmonic mean of precision and recall.
Accuracy	Proportion of all predictions that are correct.

17. Results and Screenshots

The final submission should insert actual screenshots showing application execution, dataset loading, sample detection/classification results, risk scores, explanations, generated alerts, and evaluation output. This report intentionally does not fabricate screenshots or performance values.

18. Challenges

- Separating dataset management from Java application logic.
- Designing features that represent both behaviour and data-transfer characteristics.
- Handling malformed or incomplete activity records.
- Combining anomaly evidence, classification, risk scoring, and explanation.
- Maintaining a modular structure while keeping the workflow easy to understand.

19. Learnings / Key Takeaways

- Modular architecture makes a cybersecurity application easier to develop and test.
- Preprocessing and feature engineering are important before behavioural analysis.
- Risk scores become more useful when accompanied by explanations and alerts.
- Version control and clear repository structure improve reproducibility.
- Performance metrics must come from actual experiments.

20. Future Enhancements

- Integrate real cybersecurity datasets where access and licensing permit.
- Train and evaluate a dedicated Random Forest classifier.

- Compare multiple anomaly-detection techniques experimentally.
- Add database storage and a security-analyst dashboard.
- Support near-real-time monitoring and configurable alert notifications.
- Introduce advanced explainable-AI techniques and larger-scale testing.

21. Conclusion

The project provides a structured Java-based approach for analysing user activity and identifying potential data-exfiltration behaviour. Its modular pipeline covers data loading, preprocessing, feature engineering, behaviour profiling, anomaly detection, classification, risk scoring, explainability, alert generation, and evaluation. The architecture supports the supplied academic requirements for functional modules, non-functional requirements, documentation, testing, GitHub organization, and future extension.

22. References

1. VITyarthi Build Your Own Project guidelines supplied for this project.
2. Java Platform and standard Java library documentation used by the implementation.
3. Apache Maven documentation for project build and testing.
4. JUnit documentation for automated testing.
5. Documentation for the Java machine-learning library selected in the final implementation.
6. CERT Insider Threat Dataset / CMU SEI documentation, if used.
7. CICIDS2017 / Canadian Institute for Cybersecurity documentation, if used.